Progetto di Data Mining 3.3: Classificazione testuale

Gruppo G3.3
Componenti:
Masala Gabriele 60/61/66245
Mantega Gabriele 60/61/66251
Aresu Matteo 60/99/00014

Introduzione:

Questo progetto mira a condurre un'analisi approfondita del dataset `20newgroup`,, una raccolta ampiamente utilizzata nel campo del Natural Language Processing (NLP). Il dataset contiene testi associati a messaggi di diversi newsgroup ed è suddiviso in 20 categorie tematiche. L'obiettivo primario del progetto è costruire, addestrare e confrontare più modelli di classificazione supervisionata in grado di predire correttamente il topic di un post testuale.

L'analisi si limiterà alle seguenti categorie:

- `rec.autos`
- `rec.sport.baseball`
- `sci.electronics`
- `sci.med`
- `sci.space`

Fasi del Progetto:

1. Analisi del Dataset

Caricare il dataset tramite la libreria *scikit-learn*. Il dataset può essere scaricato mediante la funzione **`fetch_20newsgroups`** (si consulti la documentazione della funzione al link https://scikit-learn.org/stable/modules/generated/sklearn.datasets.fetch_20newsgroups.html#sklearn.datasets.fetch_20newsgroups)

Caricare **separatamente** training set e test set, impostando il parametro `subset` prima a `'train'`, e poi a `'test'`, e utilizzando come valore del parametro `categories` la lista delle cinque categorie precedentemente indicate.

N.B. la funzione restituisce un oggetto contenente i dati e le informazioni sul dataset. In particolare l'oggetto avrà i seguenti attributi:

- `data:` array contenente i dati;
- `target`: array contenente le label del dataset, ovvero le 5 classi descritte in precedenza;
- `target_names`: i nomi di ciascuna classe.

Inoltre, la descrizione completa del dataset è memorizzata nell'attributo `DESCR` dell'oggetto in questione

2. Preprocessing

2.1 Tf-Idf

Ogni dato scaricato in precedenza è una stringa riportante l'intero contenuto testuale del post. Convertire ciascun documento testuale in una rappresentazione numerica utilizzando la classe `TfidfVectorizer` di scikit-learn: (si consulti la documentazione al link https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html#sklearn.feature_extraction.text.TfidfVectorizer). In particolare, impostare i seguenti parametri:

- `stop_words = 'english'` (Il parametro elimina le stopword dal testo, ovvero articoli, congiunzioni, forme verbali comuni etc.)
- `max_features = 100` (numero massimo di feature da considerare)

Applicare:

- `fit` e `transform` per trasformare il training set
- `transform` sul test set

2.2 Word Embedding

Oltre a TF-IDF, introdurre una seconda rappresentazione vettoriale tramite word embeddings pre-addestrati. Scaricare i vettori GloVe da:

https://nlp.stanford.edu/projects/glove/

Si utilizzi la versione: `glove.6B.100d.txt` (100 dimensioni). Caricare il file GloVe in Python, costruendo un dizionario in cui ogni chiave è una parola, e ogni valore è il vettore numerico associato a tale parola. Suggerimento: alla pagina https://spotintelligence.com/2023/11/27/glove-embedding, sezione "How to Use GloVe Word Embeddings In Python", si trova una possible implementazione del codice di caricamento.

Preprocessare ogni documento (tokenizzazione, trasformazione in lower case, rimozione di punteggiatura e stopword).

Rappresentare ciascun documento mediante media degli embeddings delle parole contenute. Di seguito una possibile implementazione del codice della funzione di conversione.

```python
# Glove ci dà vettori per parole. Noi dobbiamo classificare documenti.
# Facciamo la media dei vettori delle parole contenute nel post.

def get_document_vector(tokens, glove_dict):
    # Filtra le parole che sono presenti nel vocabolario del modello
    valid_words = [word for word in tokens if word in glove_dict.keys()]

    if valid_words:
        # Se ci sono parole valide, facciamo la media dei loro vettori
        vectors = glove_dict[valid_words]
        return np.mean(vectors, axis=0)
    else:
        # Se il post è vuoto o contiene parole sconosciute,
        # restituiamo un vettore di zeri
        return np.zeros(len(d.keys()))
```

Ottenere così una matrice numerica (n_samples × 100).

3. Classificazione

Utilizzando tecniche supervisionate, creare un modello di classificazione per prevedere il topic del post.

Sperimentare i seguenti algoritmi di classificazione, selezionando opportunamente i parametri principali di ciascun modello e motivando tali scelte :

- Decision Tree;
- KNN;
- MLP;
- Random Forest;
- AdaBoost;
- Extreme Gradient Boosting.

Per ognuna delle due rappresentazioni:

- Addestrare tutti i modelli
- Valutare il modello usando l'accuracy sul test set (`accuracy_score`)
- Costruire un grafico a barre (`matplotlib.pyplot.bar`) con l'accuracy ottenuta da ciascun modello.
- Confrontare sistematicamente le prestazioni TF-IDF vs Word Embedding

4. Valutazione

Oltre all'accuracy, si richiede di calcolare e riportare precision, recall, F1-score (macro). Produrre tabelle riepilogative e grafici aggiuntivi (bar plot o heatmap). Commentare l'interpretabilità dei modelli e la coerenza dei risultati tra training e test.

5. Discussione

Interpretare criticamente le prestazioni osservate, confrontando modelli, rappresentazioni dei dati e scelte di preprocessing. La discussione dovrà dare evidenza non solo del modello migliore, ma anche delle ragioni che spiegano tale comportamento.

Nella relazione finale rispondere alle seguenti domande:

- Quali differenze significative emergono tra TF-IDF e GloVe in termini di qualità della rappresentazione?
- Gli embeddings hanno catturato in modo migliore la semantica dei documenti?
- I modelli ensemble hanno gestito meglio la complessità dei dati?
- Alcune classi sono semanticamente più simili e quindi più difficili da distinguere?
- Quali approcci più avanzati potrebbero essere sperimentati?

Consegna del Progetto:

Gli studenti dovranno produrre il seguente materiale:

1. Il codice sorgente in Python.
2. Una breve relazione che spiega le scelte di preprocessing, i modelli e l'interpretazione dei risultati.
3. Una presentazione (10-15 slide max), da presentare ai docenti in una sessione dedicata, che riassuma l'intero progetto.

In dettaglio, il materiale dovrà rispettare le seguenti caratteristiche.

1. Codice Sorgente

Il codice sorgente dovrà possibilmente essere ben commentato o essere incluso in un notebook Jupyter. Fare in modo che lo script visualizzi le informazioni relative a ciascuna fase descritta in precedenza, inclusi i grafici generati.

2. Report Finale

Il report dovrà includere i seguenti paragrafi:

1. *Introduzione*. Comprende la descrizione del problema e gli obiettivi del task assegnato.
2. *Dataset*. Descrizione del dataset e delle principali caratteristiche.
3. *Preprocessing*. Descrizione dei processi di elaborazione e trasformazione del dataset effettuati, con relativa documentazione dei risultati ed eventuali grafici associati.
4. *Applicazione modelli*. Descrivere le metodologie e i modelli utilizzati, motivando eventualmente le scelte effettuate, e riportare i risultati ottenuti (compresi eventuali grafici).
5. *Discussione*. Analisi e discussione dei risultati, con un paragrafo riassuntivo che riporti le considerazioni finali sui task e sui risultati ottenuti.
6. *Conclusioni*

3. Presentazione

L'obiettivo è preparare una serie di slide (max 10-15) che riassumano e illustrino tutti i punti chiave del report. Includere una slide introduttiva e una conclusiva.

N.B. La valutazione finale del progetto sarà data in base al livello di qualità dei seguenti fattori e del relativo peso sulla valutazione finale del progetto (i pesi sono espressi in maniera indicativa)

Criterio	Descrizione	Peso %
Analisi preliminare e preprocessing	Chiarezza nella comprensione e definizione del problema , motivazione delle scelte e coerenza con gli obiettivi del corso, completezza e correttezza del flusso di preparazione dei dati (gestione dei valori mancanti, scaling, riduzione dimensionale, visualizzazioni).	15%
Scelte tecnologiche, configurazione e applicazione degli algoritmi di clustering	Adeguatezza della motivazione delle eventuali scelte relative ai parametri e ai modelli utilizzati.	15%
Analisi e interpretazione dei dati e dei risultati	Uso corretto delle metriche, interpretazione critica e discussione dei pattern emersi.	20%
Qualità del report e delle slide di presentazione	Chiarezza espositiva, struttura logica, qualità grafica e capacità di sintesi dei risultati principali.	20%
Implementazione e correttezza del codice Python	Qualità, leggibilità e organizzazione del codice; uso appropriato delle librerie (*pandas, scikit-learn, matplotlib*, ecc.); riproducibilità dei risultati.	20%
Discussione e capacità di argomentazione orale	Padronanza dei contenuti, capacità di rispondere alle domande, di spiegare le scelte metodologiche e di discutere criticamente i risultati.	10%